

USENIX

THE ADVANCED COMPUTING SYSTEMS ASSOCIATION

From Symmetry toward Weak Asymmetry: Secure Steganography under the Receiver's Partial Channel Knowledge

Guorui Liao, Shuyu Gan, Jinshuai Yang, Yongfeng Huang, and Zhongyi Huang, *Tsinghua University*

https://www.usenix.org/conference/usenixsecurity26/presentation/liao

This paper is included in the Proceedings of the 35th USENIX Security Symposium.

August 12–14, 2026 • Baltimore, MD, USA

ISBN 978-1-939133-58-8

Open access to the Proceedings of the 35th USENIX Security Symposium is sponsored by



جامعة الملك عبدالله للعلوم والتقنية

King Abdullah University of Science and Technology

ARTIFACT EVALUATED
USENIX
AVAILABLE

From Symmetry toward Weak Asymmetry: Secure Steganography under the Receiver's Partial Channel Knowledge

Guorui Liao Shuyu Gan Jinshuai Yang Yongfeng Huang Zhongyi Huang
Tsinghua University

Abstract

Provably secure steganography enables covert communication by disguising secret messages as normal communication content while guaranteeing computational indistinguishability. Recent work typically assumes that the sender can construct stegotext based on explicit conditional distributions, while the receiver either has access to the same distributions for decoding or has no such access at all. In many realistic communication settings, however, the receiver's access lies between these two extremes, a regime that is not captured by either assumption. We therefore introduce and study a new setting, called weakly asymmetric steganography, in which the sender has access to the full conditional distributions used for stegotext construction, while the receiver only has partial knowledge derived from them. We design provably secure steganography schemes under several representative forms of receiver-side knowledge. Specifically, we consider binary alphabets in which the receiver only knows whether a symbol's probability exceeds a given threshold, as well as general alphabets where the receiver only observes a possibly partial ranking of symbols by probability. We evaluate our rank-based constructions in language-model-based text steganography under both limited receiver-side knowledge and precision-induced distribution mismatch. The results show that weakly asymmetric steganography can achieve non-trivial embedding capacity, and further suggest the potential of this setting in distribution-mismatch scenarios.

1 Introduction

As surveillance and content censorship become increasingly prevalent, the right to free and private communication in cyberspace faces serious threats [12, 22]. In response, a wide range of privacy-enhancing technologies for communication have been studied and deployed. Among them, steganography circumvents censorship by concealing the very existence of secret communication.

Steganography disguises a secret message (i.e., *hiddentext*) as seemingly normal content. The resulting disguised content (i.e., *stegotext*) is designed to make it difficult for third-party observers to distinguish it from truly normal content (i.e., *covertext*). Through stegotext-based communication, the communicating parties can exchange hiddentext while external observers (censors or service providers), even with full access to all communication content, remain unable to determine whether any secret communication is taking place.

To make the above communication truly secure, researchers formally defined the security of steganography schemes as the computational indistinguishability between stegotext and covertext under the *chosen hiddentext attack* model (the steganographic version of the *chosen plaintext attack* model in cryptography). Schemes that satisfy this security definition and the correctness requirement (i.e., the ability to correctly recover the hiddentext) are referred to as **provably secure steganography** [15, 16]. Intuitively, this means that for any observer with practical computational capabilities, even if they know what the covertexts are expected to look like and even know or can choose the hiddentext to be embedded, they still cannot distinguish the stegotext constructed by the sender from genuine covertext.

For the construction of indistinguishable stegotext, recent mainstream steganography models, such as **symmetric steganography** [17], typically assume that the sender has access to an oracle which, given the current communication history, provides the explicit conditional distribution over the next covertext symbol. Based on this access, the sender can construct stegotext via a secret-driven sampling algorithm that iteratively selects symbols from the conditional distribution as a function of the hiddentext and updates the communication history until natural termination. The rapid development of AIGC (Artificial Intelligence Generated Content) further supports the practicality of this oracle assumption. In particular, autoregressive generative models [7, 13, 21, 25], such as large language models, explicitly output the conditional distribution over the next token given the current context. As a result, such models naturally instantiate the assumed oracle.

Under this common sender-side assumption, existing models of provably secure steganography can be classified by the

receiver-side access they assume. In symmetric steganography, the receiver is assumed to have the same oracle access as the sender, enabling decoding based on the explicit conditional distributions. In practice, this assumption is typically realized by having both parties deploy the same generative model. In contrast, **undetectable steganography** [26], also referred to as **asymmetric steganography** [2], assumes that the receiver has no access to such an oracle and performs decoding solely based on the received stegotext.[1]

However, the two receiver-side assumptions above and their corresponding steganography schemes fail to capture many intermediate practical scenarios. In such scenarios, the receiver may not have access to the oracle described above and therefore cannot obtain the complete conditional distributions, yet may still possess some useful information related to these distributions. For instance, in language-model-based text steganography, such intermediate scenarios can arise from two common sources. First, the receiver may be unable to obtain the full conditional distribution due to limited device capability or restricted model access. Instead, it may only interact with the model through limited APIs, which expose partial observations of the distribution, such as top-k tokens or sampled outputs. Second, even if the receiver has access to the same generative model as the sender, it may still fail to reproduce the sender's exact distributions. This can happen because of numerical precision, hardware differences, or inference nondeterminism [14]. As a result, the receiver may obtain a perturbed version of the sender's distribution rather than the exact distribution itself.

In these scenarios, symmetric steganography, which relies on precise distribution access at the receiver, may fail to decode. Asymmetric steganography avoids this requirement, but typically provides only very limited embedding capacity and ignores the partial information already available to the receiver.

To address this gap, we introduce the notion of **weakly asymmetric steganography**, which studies this intermediate regime through a clean and tractable abstraction: the receiver obtains only the information specified by a deterministic function of the sender's conditional distribution, for example, the relative ordering of symbols under that distribution. We refer to such information as the receiver's *partial channel knowledge*. Figure 1 illustrates how our setting compares to existing steganography models. This abstraction captures a tractable core of the broader intermediate regime, while also providing a starting point for studying more approximate or noisy forms of receiver-side information. For example, information inferred from repeated samples, or statistical properties that may be preserved under distributional perturbations, can be viewed as natural extensions of this abstraction.

Weakly asymmetric steganography raises several fundamental questions that have not been studied before. One of the most basic questions is whether, given a fixed form of partial channel knowledge, there exist steganography schemes that decode using only this knowledge and effectively exploit it. To the best of our knowledge, existing work provides no clear evidence that such schemes exist for any concrete form of partial channel knowledge, beyond the trivial approach of ignoring such knowledge and falling back to fully asymmetric steganography schemes.

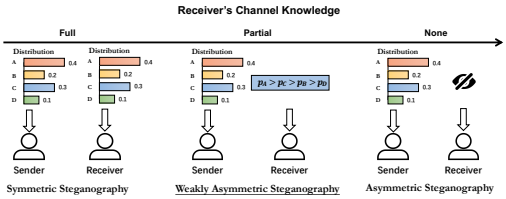


Figure 1: Comparison of symmetric, asymmetric, and weakly asymmetric steganography scenarios.

In this work, we take a first step toward answering this question by studying and constructing weakly asymmetric steganography schemes under two natural forms of receiver's partial channel knowledge: threshold-based knowledge in binary alphabets and rank-based knowledge in general alphabets. The former tells the receiver whether a symbol's probability exceeds a threshold, while the latter gives a full or partial ranking of symbols by probability. We further evaluate the proposed rank-based schemes in language-model-based text steganography under both limited receiver-side knowledge and precision-induced distribution mismatch.

Our main contributions are as follows:

1. We introduce a new formal setting, weakly asymmetric steganography, which captures scenarios where the sender has access to the full conditional distributions of the channel, while the receiver possesses only partial knowledge about these distributions.

2. We initiate the study of weakly asymmetric steganography by designing provably secure schemes under two representative forms of partial channel knowledge. In particular, we present schemes under threshold-based knowledge in binary alphabets and rank-based knowledge in general alphabets, demonstrating that partial channel knowledge can be explicitly exploited to achieve meaningful embedding capacity.

3. We empirically evaluate the proposed rank-based weakly asymmetric schemes in language-model-based text steganography. Our experiments show that weakly asymmetric steganography achieves non-trivial embedding capacity under limited receiver-side knowledge, and further suggest that this paradigm has potential in precision-induced distribution-mismatch scenarios.

[1]The terms symmetric steganography and asymmetric steganography here are distinct from symmetric-key steganography [16] and public-key steganography [23].

Organization

The rest of this paper is organized as follows. Section 2 reviews the necessary background and related work on provably secure steganography. Section 3 introduces the formal model of weakly asymmetric steganography. Section 4 presents concrete constructions of weakly asymmetric steganography schemes under threshold and rank-based channel knowledge. Section 5 reports experimental evaluations of the proposed schemes in language-model-based text steganography, including studies under partial channel knowledge and distribution mismatch.

2 Background and Related Work

Provably Secure Steganography In 2002, Hopper et al. [16] introduced provably secure steganography, formalizing two fundamental requirements: correctness and security. Correctness requires that the receiver's decoding error be negligible, while security requires that no probabilistic polynomial-time (ppt.) adversary can distinguish stegotexts from covertexts with non-negligible advantage under chosen hiddentext attacks. This paradigm was initially studied in the *universal steganography* setting, where the sender has access to an oracle that samples from the channel distribution, whereas the receiver has no access to that distribution. Several schemes based on *rejection sampling* have been proposed in this setting [1, 3, 4, 10, 20, 23]. This setting minimizes oracle requirements, while existing schemes typically depend on strong and often unrealistic assumptions. For instance, they require the channel to exhibit a positive lower bound on the entropy of its conditional distributions, or assume the existence of suitable unbiased hash functions. Such conditions are rarely satisfied in practice [17]. In addition, these schemes generally provide only limited embedding capacity, which further restricts their practical applicability.

Symmetric Steganography with Generative Models *Symmetric steganography*, formalized by Kaptchuk et al. [17], assumes that both the sender and the receiver have access to an oracle that provides the channel distribution. This setting has become one of the most widely adopted paradigms [11, 17, 19, 24] in the study of provably secure steganography, partly because it naturally aligns with modern generative models [13, 21, 25] and enables practical and efficient stegosystems. Indeed, a generative model can be viewed as a function that maps a history (or prefix) to a conditional distribution over the next symbol, thereby instantiating the oracle assumed in symmetric steganography.

In practice, the most common application scenario of symmetric steganography is text steganography based on language models. Here, the sender transforms a secret message into fluent text by leveraging the shared language model and a symmetric steganography scheme. With provable security, the generated stegotext is computationally indistinguishable from normal language-model outputs. The receiver, equipped with the same language model, can reliably decode the embedded message, while this shared-model setting also supports high embedding capacity. Recent symmetric steganography schemes [11, 24] achieve embedding rates close to the channel entropy, which is the theoretical upper bound.

However, the correctness of symmetric steganography schemes relies on a fragile consistency requirement: at each step, the receiver must reconstruct the same conditional distribution used by the sender, even though the receiver itself does not generate stegotext. In language-model-based instantiations, this requires not only that both parties can run the same model (typically an open-source model), but also that their inference pipelines match closely, since even small differences in implementation details or numerical precision can change the induced conditional distribution.

Asymmetric Steganography (Undetectable Steganography) *Undetectable watermarking*, proposed by Christ et al. [9], was originally designed for watermarking the outputs of large language models. Its core assumption is that the sender (i.e., the model owner) controls the generation process, while the receiver (i.e., the detector), lacking access to the language model and the prompt, has no access to the conditional distributions underlying the generated text. The goal is to embed a detectable watermark while ensuring that watermarked outputs remain computationally indistinguishable from normal text, thereby preserving both linguistic quality and the statistical characteristics of model outputs. Building on this line of work, subsequent research generalized the embedded signal from a single watermark bit to richer information, leading to the notion of *undetectable steganography* [26], which is also referred to as *asymmetric steganography* in later work [2].

Existing schemes [2, 8, 26] typically encode information by biasing the sender's symbol choice at each generation step based on the secret message. Although the receiver does not know the underlying channel distribution, given the secret key, each symbol has a small amount of mutual information with the secret message, allowing the receiver to recover the message by aggregating many observed symbols and performing hypothesis testing. This design is attractive because the receiver can decode solely from the observed symbols, without requiring access to the channel distribution or maintaining a synchronized channel history. Compared to symmetric steganography, the receiver does not need to run an identical and computationally expensive model as the sender, nor does it need to keep track of past context. Moreover, it is often robust to certain modifications of the generated stegotext.

However, these advantages come at a cost. In particular, the achievable embedding capacity is significantly constrained, making these schemes less suitable for covert communication scenarios that demand higher throughput. Moreover, these

schemes still require careful balancing between the embedding rate and decoding correctness, as decoding is based on hypothesis testing.

3 Definitions

3.1 Motivation

Symmetric and asymmetric steganography correspond to two extreme receiver-side assumptions. Many practical scenarios, however, lie between them: the receiver may be unable to obtain or exactly reproduce the conditional distributions, while still having useful information about them. This suggests an intermediate regime that is practically meaningful and theoretically worth studying.

Modeling such intermediate cases in full generality is difficult, since the receiver's information is often approximate, noisy, and hard to characterize uniformly in practice. We therefore adopt a clean and tractable abstraction: the receiver obtains only the information specified by a deterministic function $f(D)$, where D is the sender's conditional distribution. We call f the *knowledge function*, and refer to steganography under this abstraction as *weakly asymmetric steganography*. This abstraction directly captures settings where the receiver is given partial channel knowledge, while also providing a basis for studying broader settings. For example, such knowledge may be inferred from repeated samples. Moreover, when the receiver only has an approximate version of the sender's distribution, coarse knowledge such as ordering may be more stable than exact probabilities, and may help make decoding more reliable in such cases.

In this section, we introduce the notation used throughout this work and formalize weakly asymmetric steganography in the symmetric-key setting.

3.2 Notations

Following the formulation of provably secure steganography [15, 17], we use the term *covertext channel* to refer to the normal communication channel in the steganographic setting. We write Σ for the alphabet of symbols that may appear in the communication, and denote $\Sigma = \{\sigma_0, \ldots, \sigma_{|\Sigma|-1}\}$.

We describe the behavior of a covertext channel by a probability distribution, referred to as the *covertext channel distribution* (or simply the *channel distribution*). Intuitively, the covertext channel distribution describes how normal communication typically continues after any given history. Formally, the covertext channel distribution $\mathcal{D}$ is a probability distribution over Σ^*, where Σ^* denotes the set of all finite sequences over Σ. For any history $\mathcal{H} = (h_0, \ldots, h_{n-1}) \in \Sigma^*$, $\mathcal{D}$ induces the conditional distribution over Σ of the next symbol in normal communication given $\mathcal{H}$, which we denote by $\mathcal{D}_{\mathcal{H}}$. When unambiguous, we will simply refer to $\mathcal{D}_{\mathcal{H}}$ as *the conditional distribution*, and we will often use D to denote it.

3.3 Knowledge Function & Knowledge

We first formalize the notion of a *knowledge function*, which specifies what information about a distribution is available to a communication entity. Formally, a knowledge function is a mapping

$$f : \mathcal{X} \to \mathcal{Y},$$

where $\mathcal{X}$ denotes the set of probability distributions over the alphabet Σ, and $\mathcal{Y}$ is the corresponding feature space. For a distribution $D \in \mathcal{X}$, we refer to the value $f(D) \in \mathcal{Y}$ itself as the *knowledge*. Typical examples of knowledge functions include, for instance, the *rank* function $\mathsf{rank}(\cdot)$, which maps a distribution D over Σ to the ordering of symbols by probability.

We say that a communication entity has knowledge f about the channel distribution $\mathcal{D}$ if, for every history $\mathcal{H}$, it has access to the value $f(\mathcal{D}_{\mathcal{H}})$. For brevity, we refer to this access as having knowledge $f(\mathcal{D})$.

The identity knowledge function $f_{\mathrm{id}}(D) = D$ corresponds to *full channel knowledge*. A constant knowledge function corresponds to *no channel knowledge*, since it reveals no nontrivial information about the conditional distribution. In this work, we focus on non-degenerate knowledge functions that lie strictly between these two extremes, and we refer to this regime as having *partial channel knowledge*.

3.4 Weakly Asymmetric Steganography

We follow the formal definition of *symmetric steganography* given by Kaptchuk et al. [17] and adapt it to define *weakly asymmetric steganography*, where the receiver has only partial channel knowledge. We state the definition as follows:

Definition 1 (Weakly Asymmetric Steganography)**.** A weakly asymmetric steganography scheme $\Pi_{\mathcal{D},f}$, defined under a covertext channel distribution $\mathcal{D}$ and a knowledge function f, is a triple of possibly probabilistic algorithms,

$$\Pi_{\mathcal{D},f} = (\mathsf{KeyGen}_{\mathcal{D}}, \mathsf{Encode}^{f}_{\mathcal{D}}, \mathsf{Decode}_{f(\mathcal{D})}).$$

- $\mathsf{KeyGen}_{\mathcal{D}}(1^{\lambda})$: generates the secret key sk.
- $\mathsf{Encode}^{f}_{\mathcal{D}}(\mathsf{sk}, m, \mathcal{H})$: takes as input a secret key sk, a secret bit string m (also called the *hiddentext*), and the channel history $\mathcal{H}$. It outputs a stegotext $c \in \Sigma^*$.
- $\mathsf{Decode}_{f(\mathcal{D})}(\mathsf{sk}, c, \mathcal{H})$: takes as input a secret key sk, a stegotext c, and the channel history $\mathcal{H}$. It outputs a bit string m on success, or the empty string ε on failure.

Here, the subscript $\mathcal{D}$ indicates that the algorithm has full channel knowledge, meaning that for any history $\mathcal{H}$, it has access to the exact value of $\mathcal{D}_{\mathcal{H}}$. The superscript f indicates that the encoding algorithm also knows the public knowledge function f. In contrast, the subscript $f(\mathcal{D})$ indicates that the algorithm has access only to the knowledge specified by $f(\mathcal{D})$, that is, for any history $\mathcal{H}$, it knows only the value $f(\mathcal{D}_{\mathcal{H}})$.

Correctness. A scheme must ensure correctness, i.e., the $\mathsf{Decode}_{f(\mathcal{D})}$ algorithm can recover the embedded message from the stegotext except with negligible probability. Formally, for any secret key $\mathsf{sk} \leftarrow \mathsf{KeyGen}_{\mathcal{D}}(1^{\lambda})$:

$$\Pr[\mathsf{Decode}_{f(\mathcal{D})}(\mathsf{sk}, \mathsf{Encode}^{f}_{\mathcal{D}}(\mathsf{sk}, m, \mathcal{H}), \mathcal{H}) = m] \geq 1 - \mathsf{negl}(\lambda).$$

Security. Security requires that no ppt. adversary, given the full channel distribution $\mathcal{D}$, can distinguish oracle access to $\mathsf{Encode}^{f}_{\mathcal{D}}(\mathsf{sk}, \cdot, \cdot)$ from oracle access to a random sampling oracle $O_{\mathcal{D}}(\cdot, \cdot)$ that returns a sample generated according to $\mathcal{D}$ with the appropriate length. Formally, a scheme is secure against chosen hiddentext attacks if, for all ppt. adversaries $\mathcal{A}_{\mathcal{D}}$, with $\mathsf{sk} \leftarrow \mathsf{KeyGen}_{\mathcal{D}}(1^{\lambda})$, the following inequality holds:

$$\left| \Pr\left[\mathcal{A}_{\mathcal{D}}^{\mathsf{Encode}^{f}_{\mathcal{D}}(\mathsf{sk}, \cdot, \cdot)} = 1 \right] - \Pr\left[\mathcal{A}_{\mathcal{D}}^{O_{\mathcal{D}}(\cdot, \cdot)} = 1 \right] \right| \leq \mathsf{negl}(\lambda).$$

Informally, symmetric and asymmetric steganography can be viewed as two boundary cases of our formulation. In the symmetric setting, both the sender and the receiver have full channel knowledge. In contrast, the fully asymmetric setting can be roughly modeled by taking f to be a constant function (e.g., $f \equiv 0$), meaning that the receiver has no meaningful channel knowledge. Weakly asymmetric steganography, as studied in this work, captures the intermediate regime where f reveals some structured information about the channel distribution, while still falling short of full channel knowledge.

For any knowledge function f, every asymmetric steganography scheme is a valid weakly asymmetric steganography scheme, where the receiver can simply ignore $f(\mathcal{D})$. Therefore, the central question is not existence, but non-triviality: whether there exist schemes that meaningfully exploit the receiver's knowledge $f(\mathcal{D})$ to gain a strict advantage over simply ignoring it. We call such schemes non-trivial weakly asymmetric steganography schemes under knowledge f. In this work, we focus on such non-trivial schemes.

4 Schemes under Threshold and Rank Knowledge

A natural question regarding weakly asymmetric steganography is whether this setting has practical significance, or whether it amounts to a purely definitional generalization of symmetric and asymmetric steganography. More concretely, to the best of our knowledge, existing work has not established the existence of non-trivial weakly asymmetric steganography schemes under any form of partial channel knowledge.

This section provides an initial answer to this question by presenting constructions under two specific forms of partial channel knowledge. We consider *threshold-based knowledge* over binary alphabets, as well as *rank-based knowledge* over general alphabets, demonstrating the existence of non-trivial weakly asymmetric steganography schemes in both cases. Throughout this section, all schemes are *stateful* [6]: the encoding and decoding algorithms maintain internal states that evolve with each invocation, following standard practice in symmetric steganography [11, 17, 19, 24].[2] They are also *instantaneously decodable*: each received symbol can be decoded immediately, without relying on future symbols, which helps limit error propagation in noisy cases.

Before introducing the constructions, we recall a standard technical component used in many symmetric steganography schemes [17, 19], namely a *stateful pseudorandom generator*.

4.1 Stateful Pseudorandom Generators

Following most classical constructions in symmetric steganography, particularly Meteor [17], our schemes utilize a stateful pseudorandom generator (PRG) [5]. We follow Meteor's notation for stateful PRGs here. The stateful PRG consists of two algorithms:

$$\mathsf{PRG.Setup}, \mathsf{PRG.Next}.$$

The setup algorithm generates the initial state material for the PRG and each invocation of PRG.Next outputs fresh pseudorandom bits of sufficiently long fixed length. We interpret this output as a real value in $[0, 1)$ by normalizing the bit string in the natural way. We assume standard security: no ppt. adversary can distinguish oracle access to PRG.Next from oracle access to a truly uniform bit source with non-negligible advantage. Informally, sharing the same initial PRG state allows the sender and receiver to generate shared pseudorandomness.

4.2 Binary Alphabet under Threshold Knowledge

For a binary alphabet $\Sigma = \{\sigma_0, \sigma_1\}$, a distribution D is completely determined by the value $D(\sigma_0)$. Thus, a natural form of partial knowledge is threshold information about this value.

We define a *threshold knowledge function* $\mathbf{1}_{\theta}(\cdot)$, which indicates whether the probability of symbol σ_0 in the distribution D is below a given threshold θ. Specifically, it outputs:

$$\mathbf{1}_{\theta}(D) \;:=\; \mathbf{1}[D(\sigma_0) < \theta] = \begin{cases} 0, & \text{if } D(\sigma_0) \geq \theta, \\ 1, & \text{if } D(\sigma_0) < \theta. \end{cases}$$

This knowledge function reveals the comparison between $D(\sigma_0)$ and θ, and equivalently the comparison between $D(\sigma_1)$ and $1 - \theta$. In this section, we consider weakly asymmetric steganography under the threshold knowledge function $\mathbf{1}_{\theta}(\cdot)$.

A natural first attempt is to follow the randomness-recovery view of Meteor [17]: the (masked) hiddentext is interpreted as the randomness in $[0, 1)$ used for sampling, and each output symbol corresponds to a sampling interval. Thus, observing a symbol tells the receiver that the hiddentext lies in

[2] Alternatively, statefulness can be avoided by sampling a short cover prefix before encoding and treating it as a nonce [9, 18].

the corresponding interval, thereby reducing the uncertainty about the hiddentext and hence revealing information in the information-theoretic sense. This idea can also be applied here. For example, if the receiver observes σ_0 and knows that $D(\sigma_0) < \theta$, then it can localize the hiddentext to a known interval of length θ, and thus obtains some information about the hiddentext. However, Meteor recovers bits by extracting the common binary prefix of the inferred interval. Therefore, when $\theta > 1/2$, the inferred interval may have no nonempty common binary prefix, and this prefix-based recovery decodes no bit.

This observation suggests that threshold knowledge should be exploitable. The key is to focus on the symbol for which the receiver can certify a probability upper bound strictly smaller than 1. This intuition leads to our scheme below. We first present the special case $\theta = 0.5$, and then extend it to handle an arbitrary threshold $\theta \in (0,1)$.

4.2.1 The Special Case $\theta = 0.5$

We begin with the case $\theta = 0.5$, which captures the core ideas of our construction and provides a convenient starting point before extending to arbitrary thresholds. In this setting, the receiver only has access to the value $\mathbf{1}_{0.5}(D)$, rather than the full conditional distribution D.

Equivalently, this means that the receiver's knowledge indicates which symbol is more likely under the binary distribution. Specifically, let $k = \mathbf{1}_{0.5}(D)$. By definition, we have

$$D(\sigma_k) \geq 0.5 \geq D(\sigma_{1-k}),$$

so σ_k is more likely, while σ_{1-k} is less likely.

Next, we describe the sender's Sample algorithm and the receiver's Recovery algorithm. The Sample algorithm specifies how the symbol is sampled based on the secret message for each conditional distribution $\mathcal{D}_{\mathcal{H}}$. The Recovery algorithm specifies how the receiver, given access to $f(\mathcal{D}_{\mathcal{H}})$, recovers the embedded bits from the received symbol. The full Encode and Decode algorithms are obtained by iterating these steps, as shown in Algorithm 4 and Algorithm 5.

(1) Sample algorithm. Given the current conditional distribution $D = \mathcal{D}_{\mathcal{H}}$, the sender first computes $k = \mathbf{1}_{0.5}(D)$, so that σ_k is the more likely symbol and σ_{1-k} is the less likely one. The sender then draws a fresh pseudorandom value $r \in [0,1)$ from the shared stateful PRG and lets b denote the first bit of the remaining secret message to be embedded. It then derives

$$r' = \begin{cases} r, & \text{if } b = 0 \\ 1-r, & \text{if } b = 1 \end{cases}$$

The value r' is used to sample a symbol. Specifically, the unit interval $[0,1)$ is partitioned according to the distribution D as follows:

$$I_{\sigma_{1-k}} = [0, D(\sigma_{1-k})), \qquad I_{\sigma_k} = [D(\sigma_{1-k}), 1),$$

Accordingly, the sender outputs σ_{1-k} if $r' < D(\sigma_{1-k})$, and σ_k otherwise.

In particular, when $b = 0$, we have $r' = r$, and the above procedure reduces to sampling within the partition $(I_{\sigma_{1-k}}, I_{\sigma_k})$ using r. When $b = 1$, we have $r' = 1 - r$, which is equivalent to sampling with r within the swapped partition $(I'_{\sigma_k}, I'_{\sigma_{1-k}})$, where:

$$I'_{\sigma_k} = [0, D(\sigma_k)], \qquad I'_{\sigma_{1-k}} = (D(\sigma_k), 1).$$

An illustrative overview is provided in Fig. 2.

If the sampled symbol is σ_{1-k}, the current secret bit b is considered successfully embedded, and the Encode algorithm proceeds to the next bit in the subsequent sampling step. Otherwise, the same secret bit is reused in the next sampling step. The formal description of the Sample algorithm is given in Algorithm 1. This procedure is feasible because the sender has full channel knowledge, in particular the exact values of $D(\sigma_k)$ and $D(\sigma_{1-k})$.

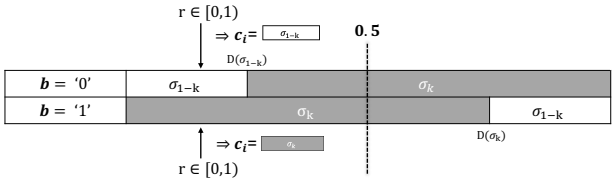


Figure 2: Illustration of the Sample algorithm under different bit b for a given pseudorandom value r. Here, c_i denotes the sampled symbol.

Algorithm 1: $\mathsf{Sample}^{\Sigma=\{\sigma_0,\sigma_1\}}_{\mathbf{1}_{0.5}}(m', \mathsf{PRG}_{\mathsf{sk}}, D)$

Input: Bits m', Pseudorandom Generator $\mathsf{PRG}_{\mathsf{sk}}$, Distribution D
Output: Sampled Symbol c_i, Number of Embedded Bits n_i

$k \leftarrow \mathbf{1}_{0.5}(D), r \leftarrow \mathsf{PRG}_{\mathsf{sk}}.\mathsf{Next}(), b = m'[0];$

$r' \leftarrow \begin{cases} r, & \text{if } b = 0 \\ 1-r, & \text{if } b = 1 \end{cases}$

if $r' < D(\sigma_{1-k})$ **then**
  $c_i \leftarrow \sigma_{1-k}, n_i \leftarrow 1;$
else
  $c_i \leftarrow \sigma_k, n_i \leftarrow 0;$
return c_i, n_i

(2) Recovery algorithm. Upon receiving the symbol and the knowledge $k = \mathbf{1}_{0.5}(D)$, the receiver first checks whether the received symbol is σ_{1-k}. If not, then no bit has been embedded in the corresponding sampling step, and hence no further decoding is required at this step.

Once the symbol σ_{1-k} is observed, the decoded bit is determined by

$$\mathbf{1}[r \geq 0.5],$$

where r is the shared pseudorandom value generated by the stateful PRG. Specifically, if $r < 0.5$, then the decoded bit is 0; otherwise, if $r \geq 0.5$, the decoded bit is 1. A formal description of the Recovery algorithm is given in Algorithm 2.

Notably, the receiver only needs to know the value of k (and hence the identity of σ_{1-k}) and the comparison between the shared randomness r and 0.5 to perform decoding, and does not require access to the exact value of $D(\sigma_{1-k})$.

Algorithm 2: $\mathsf{Recovery}^{\Sigma=\{\sigma_0,\sigma_1\}}_{\mathbf{1}_{0.5}}(c_i, \mathsf{PRG}_{\mathsf{sk}}, k)$

Input: Symbol c_i, Pseudorandom Generator $\mathsf{PRG}_{\mathsf{sk}}$, Knowledge k
Output: Bit $x \in \{0,1,\varepsilon\}$
$r \leftarrow \mathsf{PRG}_{\mathsf{sk}}.\mathsf{Next}()$;
if $c_i = \sigma_k$ **then**
  $x \leftarrow \varepsilon$;
else
  if $r < 0.5$ **then**
    $x \leftarrow 0$;
  else
    $x \leftarrow 1$;
return x

By iterating the Sample and Recovery algorithms in Algorithm 1 and Algorithm 2, we obtain the corresponding encoding and decoding algorithms $\mathsf{Encode}^f_{\mathcal{D}}$ and $\mathsf{Decode}_{f(\mathcal{D})}$, as formalized in Algorithm 4 and Algorithm 5. Together with the key generation algorithm $\mathsf{KeyGen}_{\mathcal{D}}$ in Algorithm 3, this yields the following theorem.

Theorem 1. *For any covertext channel distribution $\mathcal{D}$ over the binary alphabet $\Sigma = \{\sigma_0, \sigma_1\}$, the scheme*

$$\Pi_{\mathcal{D},f} = (\mathsf{KeyGen}_{\mathcal{D}}, \mathsf{Encode}^f_{\mathcal{D}}, \mathsf{Decode}_{f(\mathcal{D})})$$

defined by Algorithm 3, Algorithm 4, and Algorithm 5, and induced by the Sample *and* Recovery *in Algorithm 1 and Algorithm 2, is a weakly asymmetric steganography scheme under the knowledge function $f = \mathbf{1}_{0.5}(\cdot)$.*

Algorithm 3: $\mathsf{KeyGen}_{\mathcal{D}}(1^\lambda)$

Input: Security Parameter 1^λ
Output: Secret Key sk
$\mathsf{sk} \leftarrow \mathsf{PRG.Setup}(1^\lambda)$;
return sk

Algorithm 4: $\mathsf{Encode}^f_{\mathcal{D}}(\mathsf{sk}, m, \mathcal{H})$

Input: Secret key sk, Hiddentext m, history $\mathcal{H}$
Output: Stegotext c
$n \leftarrow 0, c \leftarrow \varepsilon$;
while $n < |m|$ *and* c *has not ended* **do**
  $m' = m[n:]$;
  $D \leftarrow \mathcal{D}_{\mathcal{H}}$;
  $c_i, n_i \leftarrow \mathsf{Sample}^{\Sigma}_f(m', \mathsf{PRG}_{\mathsf{sk}}, D)$;
  $c \leftarrow c \| c_i, n \leftarrow n + n_i$;
  $\mathcal{H} \leftarrow \mathcal{H} \| c_i$;
return c

Algorithm 5: $\mathsf{Decode}_{f(\mathcal{D})}(\mathsf{sk}, c, \mathcal{H})$

Input: Secret key sk, stegotext c, history $\mathcal{H}$
Output: Bit string m
$m \leftarrow \varepsilon$;
Let $c = (c_0, c_1, \ldots, c_{|c|-1})$;
for $i \leftarrow 0$ **to** $|c| - 1$ **do**
  $k \leftarrow f(\mathcal{D}_{\mathcal{H}})$;
  $x \leftarrow \mathsf{Recovery}^{\Sigma}_f(c_i, \mathsf{PRG}_{\mathsf{sk}}, k)$;
  $m \leftarrow m \| x, \mathcal{H} \leftarrow \mathcal{H} \| c_i$
return m

Proof sketch of Security. We first provide an informal intuitive proof, where we simplify by treating the random values r generated by the stateful PRG as independent and uniformly random.

For any given history $\mathcal{H}$ and fixed secret bit b, the $\mathsf{Sample}^{\Sigma=\{\sigma_0,\sigma_1\}}_{\mathbf{1}_{0.5}}$ algorithm samples according to the distribution $\mathcal{D}_{\mathcal{H}}$ using the value r'. Since r' is uniformly random regardless of whether $b = 0$ or $b = 1$, the probability of sampling σ as the stego symbol c_i is

$$\Pr_{r \leftarrow \mathsf{Unif}([0,1))}\left[c_i = \sigma \mid \mathcal{H}, b\right] = \mathcal{D}_{\mathcal{H}}(\sigma).$$

Thus, conditioned on the current history $\mathcal{H}$, each execution of $\mathsf{Sample}^{\Sigma=\{\sigma_0,\sigma_1\}}_{\mathbf{1}_{0.5}}$ has exactly the same output distribution as sampling directly from $\mathcal{D}_{\mathcal{H}}$.

Furthermore, consider any two executions of $\mathsf{Sample}^{\Sigma=\{\sigma_0,\sigma_1\}}_{\mathbf{1}_{0.5}}$. Even if the corresponding secret bits are correlated, or even identical because the same bit is reused, the values r'_0 and r'_1 remain independent, since the underlying random values r_0 and r_1 are independent. Therefore, under this assumption (i.e., treating the values r as independent and uniformly random), for any fixed secret message m and secret key sk, the output of a single query to the encoding oracle $\mathsf{Encode}^f_{\mathcal{D}}$ has the same distribution as the output of a single query to the oracle $O_{\mathcal{D}}$.

Finally, using the security of the underlying stateful PRG and the statefulness of the scheme, a standard hybrid argument

yields a formal proof of security.

Proof of Correctness. We show that the Recovery algorithm correctly recovers the bit embedded by the corresponding Sample algorithm.

First, consider the case where the sampled symbol is σ_k, the more likely symbol. In this case, no bit is successfully embedded by the Sample algorithm, and hence no decoding is required. Accordingly, the Recovery algorithm outputs ε, which is correct.

We now consider the case where the sampled symbol is σ_{1-k}, in which exactly one bit is embedded. Fig. 3 depicts the sender's sampling process from the receiver's view and makes the correctness of the recovery rule intuitive. In the left half of the diagram, where $0 \le r < 0.5$, the symbol σ_{1-k} may be sampled when $b = 0$, but it cannot be sampled when $b = 1$. Therefore, if the shared pseudorandom value $r < 0.5$, then sampling σ_{1-k} implies that the embedded bit must be 0. Equivalently,

$$r < 0.5 \ \wedge \ c_i = \sigma_{1-k} \implies \text{embedded bit} = 0.$$

Indeed, if the embedded bit were 1, the Sample algorithm would output σ_k when $r < 0.5$, since $D(\sigma_k) \ge 0.5$, contradicting the sampling of σ_{1-k}. Similarly, when $r \ge 0.5$, sampling σ_{1-k} implies that the embedded bit must be 1. That is,

$$r \ge 0.5 \ \wedge \ c_i = \sigma_{1-k} \implies \text{embedded bit} = 1.$$

Therefore, in this case the embedded bit can be expressed as

$$\mathbf{1}[r \ge 0.5],$$

which coincides exactly with the decoding rule used by the Recovery algorithm.

Therefore, in all cases, the Recovery algorithm correctly recovers the embedded bit. Since the Encode and Decode algorithms are obtained by iterating the corresponding Sample and Recovery procedures, the correctness of the scheme follows. If the stegotext naturally terminates before the entire hiddentext is embedded, correctness is understood with respect to the embedded prefix of the hiddentext.

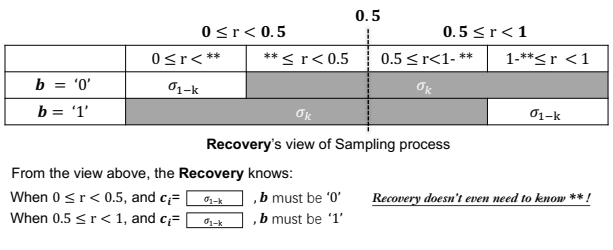


Figure 3: Illustration of the recovery procedure. The marker "**" indicates that the exact value of $D(\sigma_{1-k})$ is unknown to the receiver.

Capacity. We note that the Recovery algorithm can recover exactly 1 bit if and only if the sampled symbol is σ_{1-k}, which occurs with probability $D_{\min} = D(\sigma_{1-k}) \le 0.5$. Therefore, the capacity (expectation) for this symbol is

$$C_{\mathbf{1}_{0.5}} = 1 \times D_{\min} + 0 \times (1 - D_{\min}) = D_{\min}.$$

4.2.2 General Case $\theta \in (0,1)$

To generalize the threshold from $\theta = 0.5$ to an arbitrary $\theta \in (0,1)$, we first reinterpret the Sample and Recovery for the case $\theta = 0.5$.

The sender first applies a bit-controlled transformation to the shared random value r according to the current secret bit b (i.e., choosing to use either r or $1-r$), and then samples according to the distribution D. Moreover, a secret bit is considered successfully embedded and the sender proceeds to the next bit, if and only if the receiver is able to uniquely decode this bit. On the receiver side, decoding is attempted only upon receiving a symbol whose probability admits a known upper bound (i.e. σ_{1-k}). Furthermore, decoding is performed only when the value r lies in a region that allows the secret bit to be uniquely determined. In the case $\theta = 0.5$, as discussed above, this upper bound equals 0.5, and the corresponding decodable region covers the entire interval $[0,1)$. From this perspective, extending the construction to a general threshold θ reduces to identifying the range of random values for which the receiver can uniquely decode.

On the sender side, the sampling procedure remains unchanged: given the current secret bit b, the sender applies the same bit-controlled transformation to the shared random value, namely using $r' = r$ if $b = 0$ and $r' = 1 - r$ if $b = 1$, and then samples a symbol according to the distribution D. Consequently, for a general θ, the range of random values for which the receiver can uniquely decode is

$$[0, \min(\theta, 1-\theta)] \cup (\max(\theta, 1-\theta), 1)$$

This is because, under the receiver's knowledge $k = \mathbf{1}_\theta(D)$, we have

$$D(\sigma_k) \ge \min(\theta, 1-\theta) \quad \text{and} \quad D(\sigma_{1-k}) \le \max(\theta, 1-\theta).$$

Therefore, when $r \le \min(\theta, 1-\theta)$, observing the symbol σ_{1-k} necessarily implies that it was sampled using $r' = r$, and hence the embedded bit can be uniquely determined to be 0. Similarly, when $r > \max(\theta, 1-\theta)$, the embedded bit can only be 1.

A corresponding intuitive illustration is given in Figure 4. In the left region $0 \le r \le \min(\theta, 1-\theta)$, the symbol σ_{1-k} may be sampled when $b = 0$, but cannot be sampled when $b = 1$; hence observing σ_{1-k} uniquely determines the embedded bit as 0. Symmetrically, in the right region $\max(\theta, 1-\theta) < r < 1$, the symbol σ_{1-k} may be sampled when $b = 1$, but cannot be sampled when $b = 0$; hence observing σ_{1-k} uniquely determines the embedded bit as 1. The middle region remains

ambiguous from the receiver's viewpoint and is therefore treated as non-decodable.

Algorithm 6: $\mathsf{Sample}_{\mathbf{1}_\theta}^{\Sigma=\{\sigma_0,\sigma_1\}}(m', \mathsf{PRG}_{\mathsf{sk}}, D)$

Input: Bits m', Pseudorandom Generator $\mathsf{PRG}_{\mathsf{sk}}$, Distribution D
Output: Symbol c_i, Number of Embedded Bits n_i
$k \leftarrow \mathbf{1}_\theta(D), r \leftarrow \mathsf{PRG}_{\mathsf{sk}}.\mathsf{Next}(), b = m'[0];$
$r' \leftarrow \begin{cases} r, & \text{if } b = 0 \\ 1-r, & \text{if } b = 1 \end{cases}$
if $r' < D(\sigma_{1-k})$ **then**
  $c_i \leftarrow \sigma_{1-k}$;
  if $\min(1-\theta,\theta) < r \le \max(1-\theta,\theta)$ **then**
    $n_i \leftarrow 0$;
  else
    $n_i \leftarrow 1$;
else
  $c_i \leftarrow \sigma_k, n_i \leftarrow 0$;
return c_i, n_i

Algorithm 7: $\mathsf{Recovery}_{\mathbf{1}_\theta(D)}^{\Sigma=\{\sigma_0,\sigma_1\}}(c_i, \mathsf{PRG}_{\mathsf{sk}}, k)$

Input: Symbol c_i, Pseudorandom Generator $\mathsf{PRG}_{\mathsf{sk}}$, Knowledge k
Output: Bit $x \in \{0,1,\varepsilon\}$
$r \leftarrow \mathsf{PRG}_{\mathsf{sk}}.\mathsf{Next}();$
if $c_i = \sigma_k$ **then**
  $x \leftarrow \varepsilon$;
else
  if $r \le \min(1-\theta,\theta)$ **then**
    $x \leftarrow 0$;
  else
    if $r > \max(1-\theta,\theta)$ **then**
      $x \leftarrow 1$;
    else
      $x \leftarrow \varepsilon$;
return x

We now directly present the corresponding Sample and *Recovery* algorithms in Algorithms 6 and 7. Consistent with the above analysis, it is straightforward to verify that the resulting scheme is both secure and correct, and thus constitutes a weakly asymmetric steganography scheme under knowledge function $f = \mathbf{1}_\theta(\cdot)$. The capacity of this scheme is given by

$$C_{\mathbf{1}_\theta} = \min\big(D(\sigma_{1-k}),\, \theta,\, 1-\theta\big).$$

Since $D(\sigma_k) \ge \min(\theta, 1-\theta)$, we have

$$C_{\mathbf{1}_\theta} = \min\big(D(\sigma_{1-k}),\, D(\sigma_k),\, \theta,\, 1-\theta\big) = \min\big(D_{\min},\, \theta,\, 1-\theta\big),$$

where $D_{\min} := \min\{D(\sigma_0), D(\sigma_1)\}$.

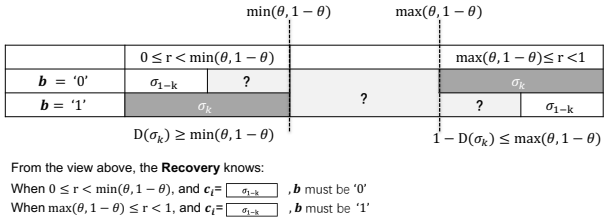


Figure 4: Illustration of the recovery procedure.

4.3 Scheme Design under Rank Knowledge

In this section, we study scheme design under rank-based knowledge over general alphabets $\Sigma = \{\sigma_0, \ldots, \sigma_{|\Sigma|-1}\}$.

In this setting, the receiver's knowledge consists of an ordering of symbols induced by the conditional distribution. We begin with the case of *full rank knowledge*, where the receiver has knowledge of the complete ordering over the alphabet. Formally, this is represented by a knowledge function $f = \mathsf{rank}(\cdot)$ which maps a distribution D over the alphabet Σ to a permutation

$$\pi = \mathsf{rank}(D),$$

such that

$$D(\sigma_{\pi(0)}) \ge D(\sigma_{\pi(1)}) \ge \cdots \ge D(\sigma_{\pi(|\Sigma|-1)}).$$

We write $\Sigma' = \pi(\Sigma)$ to denote the list Σ ordered according to π, where $\Sigma'[i] = \sigma_{\pi(i)}$.

We then extend the construction to *partial rank knowledge*, where the receiver only observes a restricted ordering over a subset of symbols. In this setting, the receiver's knowledge is represented by

$$\mathsf{PRank}(D) = (\hat{\pi}, \hat{\Sigma}),$$

where $\hat{\Sigma} \subseteq \Sigma$ denotes the subset of symbols whose relative order is known, and $\hat{\pi}$ specifies the ordering over $\hat{\Sigma}$.

4.3.1 Full Rank Knowledge

Our basic idea is to recursively partition the alphabet Σ into two subsets whose total probability masses admit a relative ordering that can be inferred solely from rank knowledge, ensuring that at each recursive step the receiver can identify the higher-probability subset without accessing exact probability values. Consequently, we can recursively apply the sampling and recovery procedures for threshold knowledge with $\theta = 0.5$, namely $\mathsf{Sample}_{\mathbf{1}_{0.5}}$ and $\mathsf{Recovery}_{\mathbf{1}_{0.5}}$, to perform encoding and decoding over the two subsets. The recursion continues until a single symbol is selected on the sender side, or until all embedded bits have been recovered on the receiver side.

The partitioning rule is simple. For any list Σ' sorted in descending order of probability, we always have

$$\sum_{i \bmod 2=0}^{i<|\Sigma'|} D(\Sigma'[i]) \geq \sum_{i \bmod 2=1}^{i<|\Sigma'|} D(\Sigma'[i]).$$

Therefore, splitting Σ' by parity, as specified in Algorithm 8, induces two subsets whose total probability masses have a known ordering determined purely by rank information.

Algorithm 8: $\mathsf{ParitySplit}(\Sigma')$

Input: A sorted list Σ'
Output: Two sublists (σ'_0, σ'_1)
$\sigma'_0 \leftarrow \Sigma'[0::2]$ `// even-indexed elements;`
$\sigma'_1 \leftarrow \Sigma'[1::2]$ `// odd-indexed elements;`
return (σ'_0, σ'_1)

The correctness of the scheme follows from the correctness of the weakly asymmetric scheme under the knowledge function $f = \mathbf{1}_{0.5}$, which ensures correct bit recovery at each recursive step, while rank knowledge and the deterministic partitioning rule guarantee that the decoder's recursion is fully synchronized with the sender's. For security, an intuitive explanation is as follows. At each recursive step, the probability that the $\mathsf{Sample}_{\mathbf{1}_{0.5}}$ selects a given subset is exactly equal to the total probability mass of that subset under the original distribution. As a result, the recursive procedure preserves the original distribution over symbols, and for any history $\mathcal{H}$, secret bits m' and any symbol $\sigma \in \Sigma$, we still have

$$\Pr_{r_0, r_1, \ldots \leftarrow \mathsf{Unif}[0,1)} \left[c_i = \sigma \mid \mathcal{H}, m' \right] = \mathcal{D}_{\mathcal{H}}(\sigma).$$

The corresponding $\mathsf{Sample}^{\Sigma}_{\mathsf{rank}}$ and $\mathsf{Recovery}^{\Sigma}_{\mathsf{rank}}$ are formalized in Algorithm 9 and Algorithm 10.

Capacity. At each recursive step, one bit is embedded if and only if the procedure descends into the lower-probability subset, as dictated by the behavior of the $\mathsf{Sample}_{\mathbf{1}_{0.5}}$ algorithm. Consequently, for any symbol $\sigma_{\pi(i)}$, the total number of embedded bits equals the number of times the lower-probability subset is selected along the recursion, which is exactly the number of ones in the binary representation of i.

Let $p_{\pi(i)} = D(\sigma_{\pi(i)})$ denote the probability of the i-th symbol in descending order. The expected embedding capacity is therefore

$$C_{\mathsf{rank}} = \sum_{i=0}^{|\Sigma|-1} p_{\pi(i)} \cdot \mathsf{count}_1\big(\mathsf{bin}(i)\big),$$

where $\mathsf{count}_1(\mathsf{bin}(i))$ denotes the number of ones in the binary representation of i.

This capacity admits the following bounds:

$$1 - p_{\max} \leq C_{\mathsf{rank}} \leq H(D),$$

where $p_{\max} = \max_{\sigma \in \Sigma} D(\sigma)$ and $H(D)$ denotes the entropy of D. The lower bound follows from the fact that every index $i \geq 1$ contains at least one one-bit. The upper bound holds since for all i,

$$\mathsf{count}_1(\mathsf{bin}(i)) \leq \log_2(i+1) \leq \log_2 \frac{1}{p_{\pi(i)}},$$

implying that the number of embedded bits per symbol does not exceed its information content.

Algorithm 9: $\mathsf{Sample}^{\Sigma}_{\mathsf{rank}}(m', \mathsf{PRG}_{\mathsf{sk}}, D)$

Input: Bits m', Pseudorandom Generator $\mathsf{PRG}_{\mathsf{sk}}$, Distribution D
Output: Symbol c_i, Number of Embedded Bits n_i
$n_i \leftarrow 0, \pi \leftarrow \mathsf{rank}(D), \Sigma' \leftarrow \pi(\Sigma)$ `// descending;`
while $|\Sigma'| > 1$ **do**
  $(\sigma'_0, \sigma'_1) \leftarrow \mathsf{ParitySplit}(\Sigma')$;
  $D'(\sigma'_0) \leftarrow \dfrac{\sum_{x \in \sigma'_0} D(x)}{\sum_{x \in \Sigma'} D(x)}$;
  $D'(\sigma'_1) \leftarrow \dfrac{\sum_{x \in \sigma'_1} D(x)}{\sum_{x \in \Sigma'} D(x)}$;
  $\Sigma', b \leftarrow \mathsf{Sample}^{\Sigma=\{\sigma'_0, \sigma'_1\}}_{\mathbf{1}_{0.5}}(m'[n_i:], \mathsf{PRG}_{\mathsf{sk}}, D')$;
  $n_i \leftarrow n_i + b$;
$c_i \leftarrow \Sigma'[0]$;
return c_i, n_i

Algorithm 10: $\mathsf{Recovery}^{\Sigma}_{\mathsf{rank}}(c_i, \mathsf{PRG}_{\mathsf{sk}}, \pi)$

Input: Symbol c_i, Pseudorandom Generator PRG_{sk}, Rank Knowledge π
Output: Bits x
$x \leftarrow \varepsilon, \Sigma' \leftarrow \pi(\Sigma)$ `// descending;`
while $|\Sigma'| > 1$ **do**
  $(\sigma'_0, \sigma'_1) \leftarrow \mathsf{ParitySplit}(\Sigma')$;
  $k = 0$; // $\Pr(\sigma'_0) \geq \Pr(\sigma'_1)$
  $c'_i \leftarrow \begin{cases} \sigma'_0, & \text{if } c_i \in \sigma'_0 \\ \sigma'_1, & \text{if } c_i \in \sigma'_1 \end{cases}$;
  $x \leftarrow x \| \mathsf{Recovery}^{\Sigma=\{\sigma'_0, \sigma'_1\}}_{\mathbf{1}_{0.5}}(c'_i, \mathsf{PRG}_{\mathsf{sk}}, k), \Sigma' \leftarrow c'_i$;
return x

4.3.2 Partial Rank Knowledge

We next consider *partial rank knowledge* $\mathsf{PRank}(\cdot)$, where the receiver only observes an ordering $\hat{\pi}$ over a subset of symbols $\hat{\Sigma} \subseteq \Sigma$.

Our key idea is simple: information is embedded only when the sampled symbol lies in $\hat{\Sigma}$. Otherwise, the sender samples a symbol outside $\hat{\Sigma}$ without embedding any bit. Concretely, the

sender first draws a fresh random value $r \leftarrow \mathrm{Unif}[0,1)$ (which is *not* shared with the receiver) and compares it with the total probability mass

$$p_{\hat{\Sigma}} := \sum_{\sigma \in \hat{\Sigma}} D(\sigma).$$

If $r > p_{\hat{\Sigma}}$, the sender outputs a symbol from $\Sigma \setminus \hat{\Sigma}$ according to the distribution D restricted to and renormalized on $\Sigma \setminus \hat{\Sigma}$, and no bit is embedded. Otherwise, the sender renormalizes D onto $\hat{\Sigma}$ to obtain $D_{\hat{\Sigma}}$, and applies the full-rank scheme $\mathsf{Sample}^{\hat{\Sigma}}_{\mathsf{rank}}$ on the restricted alphabet $\hat{\Sigma}$ to embed bits and sample a symbol.

If the received symbol $c_i \notin \hat{\Sigma}$, the Recovery outputs ε; otherwise, it decodes using the full rank recovery $\mathsf{Recovery}_{\mathsf{rank}}$ under knowledge $\hat{\pi}$. The corresponding sampling and recovery procedures are given in $\mathsf{Sample}_{\mathsf{PRank}}$ (Algorithm 11) and $\mathsf{Recovery}_{\mathsf{PRank}}$ (Algorithm 12). Correctness and security of the resulting scheme follow directly from those of the full-rank construction. Top-k knowledge arises as a special case by restricting $\hat{\Sigma}$ to the top-k symbols.

Algorithm 11: $\mathsf{Sample}^{\Sigma}_{\mathsf{PRank}}(m', \mathsf{PRG}_{\mathsf{sk}}, D)$

Input: Bits m', Pseudorandom Generator $\mathsf{PRG}_{\mathsf{sk}}$, Distribution D
Output: Symbol c_i, Number of Embedded Bits n_i
$(\hat{\pi}, \hat{\Sigma}) \leftarrow \mathsf{PRank}(D)$;
$r \leftarrow \mathrm{Unif}[0,1), p_{\hat{\Sigma}} \leftarrow \sum_{\sigma \in \hat{\Sigma}} D(\sigma)$;
if $r > p_{\hat{\Sigma}}$ **then**
  Sample $c_i \sim \mathsf{Normalize}(D, \Sigma \setminus \hat{\Sigma})$;
  $n_i \leftarrow 0$
else
  $D_{\hat{\Sigma}} \leftarrow \mathsf{Normalize}(D, \hat{\Sigma})$
  $c_i, n_i \leftarrow \mathsf{Sample}^{\hat{\Sigma}}_{\mathsf{rank}}(m', \mathsf{PRG}_{\mathsf{sk}}, D_{\hat{\Sigma}})$
return c_i, n_i

Algorithm 12: $\mathsf{Recovery}^{\Sigma}_{\mathsf{PRank}}(c_i, \mathsf{PRG}_{\mathsf{sk}}, (\hat{\pi}, \hat{\Sigma}))$

Input: Symbol c_i, Pseudorandom Generator $\mathsf{PRG}_{\mathsf{sk}}$, Partial Rank Knowledge $(\hat{\pi}, \hat{\Sigma})$
Output: Bits x
if $c_i \notin \hat{\Sigma}$ **then**
  return ε
else
  $x = \mathsf{Recovery}^{\hat{\Sigma}}_{\mathsf{rank}}(c_i, \mathsf{PRG}_{\mathsf{sk}}, \hat{\pi})$
return x

5 Experiments

In this section, we evaluate our rank-based weakly asymmetric steganography schemes in language-model-based text steganography.

We conduct two sets of experiments. First, we consider the setting directly captured by our model, where the receiver is given only rank-based knowledge. In particular, we evaluate both full-rank knowledge and partial-rank knowledge, where the receiver only observes the ordering of the top-k tokens.

Second, we study a distribution-mismatch setting in which the sender and the receiver run the same language model under different numerical precisions. This setting is outside the formal model, but is motivated by the approximate and noisy forms of receiver-side information discussed earlier, where decoding based on coarse knowledge such as ordering may be less sensitive to perturbations than decoding based on exact probability values.

5.1 Receiver with Partial Channel Knowledge

Setup For the channel distribution, we use two open-source language models: Meta-Llama-3-8B-Instruct[3] and Qwen2.5-7B-Instruct[4]. The sender accesses the conditional distribution by running the language model directly. To simulate the receiver's limited channel knowledge in the weakly asymmetric setting, we compute the full conditional distribution using the same language model, but provide the receiver only with the corresponding rank-based knowledge for decoding. We fix the prompt "Tell me something about steganography" as the history. Each stegotext is generated as a response to this prompt, and generation terminates when the language model produces an end-of-sequence token.

As baselines, we include asymmetric steganography and random sampling. Symmetric steganography schemes are not included as baselines, since they require full distribution access at the receiver and thus fall outside the weakly asymmetric setting. For asymmetric steganography, we use the Undetectable Steganography scheme [26], based on the official implementation[5], with the default threshold parameter $t = 2$. To avoid penalizing this baseline due to implementation inefficiencies, we apply minor implementation-level optimizations while preserving the original algorithmic behavior. These optimizations affect only runtime efficiency and do not change the reported non-runtime evaluation metrics. Random sampling is implemented using `torch.multinomial`.

We evaluate both the full-rank scheme and partial-rank schemes. For partial-rank knowledge, the receiver is restricted to the relative ordering of only the top-2, top-5, or top-10 tokens. The stateful PRG used in all rank-based weakly asymmetric schemes follows the same construction as in the classical Meteor scheme [17] and is implemented via HMAC-DRBG, with a precision of 52 bits.

All experiments are conducted on a single RTX 5880 Ada GPU with 48GB VRAM. For each experimental configuration, we generate 1,000 responses. The hiddentext and key are

[3] https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct
[4] https://huggingface.co/Qwen/Qwen2.5-7B-Instruct
[5] https://github.com/OrZamir/steg

independently re-sampled for each response. The decoding is performed directly at the token level, without converting tokens back to text, in order to avoid ambiguities introduced by text-level representations.

Metrics. *Capacity.* We report the average number of embedded bits per response (**Capacity**), the average empirical entropy per response (**Emp. Entropy**), computed as the sum of token-wise entropies along the Encode process, and the utilization ratio (**Util.**), defined as **Capacity** divided by **Emp. Entropy** [11]. *Time.* We measure the average encoding and decoding time per response (**Enc. Time** and **Dec. Time**), which include both language model inference and steganographic operations. *Other metrics.* We also report the average length of the generated stegotext in number of tokens (**Length**) and the decoding correctness (**Correct.**), measured as the fraction of responses that are decoded correctly.

Results and Analysis. Table 1 summarizes the experimental results. A concrete example of generated stegotext under full-rank knowledge is provided in Appendix A. Rank-based weakly asymmetric steganography incurs only modest additional computational overhead compared to random sampling in our experiments. Across both language models, the generated text length and empirical entropy remain similar to those of random sampling, which is consistent with the proposed schemes preserving the natural generation behavior of the underlying language models.

The results also highlight a clear capacity gap between rank-based weakly asymmetric steganography and the fully asymmetric baseline. Undetectable steganography embeds only a few bits per response, whereas our rank-based schemes embed tens to hundreds of bits per response. In particular, even top-2 ordering information already brings a meaningful capacity gain, suggesting that intermediate receiver-side channel knowledge can be useful.[6]Overall, these results support the study of weakly asymmetric steganography as a meaningful setting between symmetric and fully asymmetric steganography.

5.2 Receiver under Distribution Mismatch

The previous subsection evaluates the setting where the receiver is explicitly given limited rank-based channel knowledge. We now consider a related but different scenario motivated by precision-induced distribution mismatch, where the receiver may obtain a perturbed version of the sender's conditional distribution rather than the exact distribution itself.

In symmetric steganography, decoding correctness typically requires the receiver to reconstruct the same conditional distributions used by the sender at each generation step. Numerical discrepancies between the sender-side and receiver-side distributions may therefore lead to decoding errors. In contrast, weakly asymmetric steganography decodes from coarse knowledge such as ordering, rather than exact probability values. As a result, as long as this knowledge remains consistent, decoding can remain reliable even when the underlying distribution changes. From this perspective, weakly asymmetric steganography can be viewed as a steganography model with improved numerical robustness.

Thus, we consider a language-model-based text steganography setting in which the sender and receiver run the same language model under different numerical precisions. Specifically, the sender uses float32 precision, while the receiver performs inference and decoding under float16 precision, inducing a perturbation to the channel distribution. This setting is practically motivated, as float16 inference significantly reduces memory consumption and is commonly adopted under resource constraints.

Setup The experimental setup largely follows that of the previous subsection. We use the same language models, prompt, number of trials, and experimental hardware as before. Here, we focus only on the differences.

Unlike the standard setting where the sender and the receiver access identical channel distributions, we introduce a numerical asymmetry between the two parties. Specifically, the sender obtains the channel distribution by running the language model under float32 precision, while the receiver performs inference and decoding using the same model under float16 precision. For the asymmetric baseline, we reuse the results reported in Table 1, as its decoding does not depend on receiver-side channel distributions. In addition to asymmetric and random-sampling baselines, we include several symmetric steganography schemes for comparison, including Meteor (16-bit precision) [17], Discop (Huffman) [11], and SparSamp (32-bit) [24]. For fairness, all schemes use pseudorandomness generated via the same HMAC-DRBG construction.

To make token-level accuracy and edit distance more informative in the distribution-mismatch experiments, we use a simple fixed-budget PRG synchronization rule for our proposed schemes. In our rank-based schemes, different tokens may consume different numbers of PRG outputs. Under distribution mismatch, a local token-level error can therefore desynchronize the PRG states and cause cascading errors in later tokens.

To isolate such local errors, we fix the number of PRG outputs consumed per generated token: both sampling and recovery consume the same budget, discarding unused outputs when necessary. The budget is set to the maximum number of PRG outputs required to process one token in the corresponding implementation. This rule does not change the sampling or recovery decisions; it only fixes the PRG advancement schedule, so that token accuracy and edit distance better re-

[6]For the embedding capacities of symmetric steganography schemes, see Table 2.

Model	Scheme	Capacity Metrics			Time Metrics		Length	Correct.↑
		Capacity (bit/resp.)↑	Emp. Entropy (bit/resp.)	Util.↑	Enc. Time (s/resp.)↓	Dec. Time (s/resp.)↓		
LLAMA 3 $\lvert\Sigma\rvert \approx 128$k	Random Sampling	–	392.681	–	18.798	18.385	497.273	–
	Undet. ($t = 2$) [26]	2.530	388.932	0.65%	21.507	0.143	494.819	61.8%
	Rank (Full)	111.223	391.070	28.44%	20.662	19.115	497.441	100.0%
	Rank (Top10)	95.067	386.317	24.61%	19.115	18.699	492.808	100.0%
	Rank (Top5)	83.374	392.303	21.25%	19.215	18.884	498.172	100.0%
	Rank (Top2)	49.855	390.106	12.78%	18.915	18.753	495.921	100.0%
QWEN 2.5 $\lvert\Sigma\rvert \approx 152$k	Random Sampling	–	589.486	–	18.610	18.137	520.575	–
	Undet. ($t = 2$)	2.305	590.922	0.39%	25.384	0.171	519.993	61.6%
	Rank (Full)	170.589	588.066	29.01%	20.486	18.815	518.845	100.0%
	Rank (Top10)	142.931	594.132	24.06%	19.132	18.702	522.571	100.0%
	Rank (Top5)	118.122	597.423	19.77%	19.147	18.808	525.972	100.0%
	Rank (Top2)	63.302	590.385	10.72%	18.748	18.580	521.248	100.0%

Table 1: Experimental results for rank-based weakly asymmetric steganography under partial channel knowledge in language-model-based text steganography.

flect local decoding errors. Response-level accuracy is largely unaffected, since exact response recovery already fails once any embedded-token error occurs.[7]

Metrics. Unless otherwise specified, we use the same evaluation metrics as in the previous subsection. Here, we additionally report the following correctness metrics.

Correctness Metrics. **Token Acc.** measures the fraction of tokens for which the decoded bits match the embedded bits. **Resp. Acc.** measures the fraction of responses for which token-level decoding is correct at every generated token. **ED** reports the average edit distance between the decoded bitstring and the embedded bitstring, i.e., the minimum number of single-bit insertions, deletions, and substitutions required to transform the decoded bitstring into the embedded bitstring. For each response, we compute ED using `Levenshtein.distance` from the Python `Levenshtein` package.

Results and Analysis. Table 2 reports the experimental results. This setting is designed to evaluate how different steganography schemes trade off embedding capacity and decoding robustness when the receiver cannot accurately recover the conditional distributions, and is particularly challenging for schemes that rely on precise numerical agreement at the receiver.

While symmetric schemes typically achieve high embedding capacity and entropy utilization, their correctness degrades substantially when precise numerical distributions are unavailable. In particular, several symmetric schemes exhibit lower token-level accuracy, larger edit distance, and near-zero response-level decoding success, indicating a strong dependence on accurate probability values. Non-instantaneous schemes, such as SparSamp, are especially sensitive to such distributional deviations, since decoding errors may accumulate across sampling steps.[8]

In contrast, rank-based weakly asymmetric steganography sacrifices part of the embedding capacity in exchange for significantly improved robustness. Weakly asymmetric schemes maintain token-level accuracy above 99% across all rank settings, together with substantially lower edit distance. Moreover, in our experiments, these schemes embed substantially more bits per response than the fully asymmetric baseline while retaining meaningful response-level decoding accuracy.[9] This robustness stems from the fact that decoding depends only on rank-based channel knowledge, which is generally more stable than exact probability values under numerical perturbations. The combination of high token-level accuracy and low edit distance further suggests that rank-based weakly asymmetric steganography has the potential to achieve stronger response-level reliability with lightweight

[7] Discop may also suffer from error propagation once synchronization is lost. We do not apply the same fixed-budget rule to Discop because its Huffman-tree procedure may use many PRG outputs for some tokens, so padding every token to the worst case would introduce substantial overhead in our implementation.

[8] The only response-level success for SparSamp corresponds to a short-output case, where generation terminates before any bit is embedded; hence both the embedded and decoded bitstrings are empty.

[9] Response-level correctness should be interpreted with care, since it averages over responses with different lengths and different numbers of embedded bits; exact recovery is generally harder for longer responses or responses carrying more bits.

Model	Type	Scheme	Capacity Metrics			Time Metrics		Length	Correct. Metrics		
			Embed. (bit/resp.)↑	Entropy (bit/resp.)	Util.↑	Enc. Time (s/resp.)↓	Dec. Time (s/resp.)↓		Token Acc.↑	Resp. Acc.↑	ED↓
LLAMA 3 $\lvert\Sigma\rvert \approx 128$k	R.	Random.	–	391.773	–	19.007	9.979	499.007	–	–	–
	S.	Meteor	207.9	385.581	53.9%	19.210	10.566	496.397	99.10%	1.8%	7.439
	S.	Discop(Huff.)	304.6	387.701	78.6%	40.117	31.297	495.700	88.54%	0.9%	40.848
	S.	Sparsamp	353.6	387.782	91.2%	18.942	10.361	496.045	–	0.1%	218.029
	A.	Undet. ($t = 2$)	2.5	388.932	0.65%	21.507	0.143	494.819	–	61.8%	–
	W-A.	**Rank (Full)**	111.6	391.291	28.5%	20.576	10.518	498.091	99.69%	21.9%	1.888
	W-A.	**Rank (Top10)**	96.0	389.432	24.7%	19.321	10.390	497.888	99.81%	39.3%	0.984
	W-A.	**Rank (Top5)**	82.5	386.481	21.3%	19.191	10.348	496.653	99.85%	48.3%	0.775
	W-A.	**Rank (Top2)**	49.6	389.758	12.7%	19.068	10.366	497.812	99.86%	50.6%	0.668
QWEN 2.5 $\lvert\Sigma\rvert \approx 152$k	R.	Random.	–	589.813	–	18.739	9.708	519.851	–	–	–
	S.	Meteor	311.8	577.263	54.0%	18.683	10.206	512.867	98.72%	0.3%	10.535
	S.	Discop(Huff.)	481.3	587.834	81.9%	46.471	36.788	518.430	79.04%	0.0%	83.220
	S.	Sparsamp	550.1	598.026	92.0%	18.939	10.284	525.455	–	0.0%	399.092
	A.	Undet. ($t = 2$)	2.3	590.922	0.39%	25.384	0.171	519.993	–	61.6%	–
	W-A.	**Rank (Full)**	171.7	588.504	29.2%	20.377	10.356	517.914	99.48%	9.2%	3.367
	W-A.	**Rank (Top10)**	142.4	590.094	24.1%	18.980	10.216	519.504	99.68%	18.8%	1.826
	W-A.	**Rank (Top5)**	117.2	593.762	19.7%	18.965	10.256	522.688	99.76%	28.8%	1.316
	W-A.	**Rank (Top2)**	62.8	587.200	10.7%	18.652	10.158	518.992	99.83%	40.2%	0.875

Table 2: Experimental results under distribution mismatch (float32 sender vs. float16 receiver) for symmetric, asymmetric, and rank-based weakly asymmetric steganography in language-model-based text steganography.

error-correction mechanisms.

Overall, these results further support the potential of weakly asymmetric steganography in settings where the receiver cannot reliably recover the exact conditional distributions.

6 Discussion

Real-world applicability. Weakly asymmetric steganography targets the intermediate regime between symmetric and fully asymmetric steganography. Its most direct application is when the receiver's capability is explicitly restricted to a given form of knowledge, for example when an interface exposes only limited features of the underlying distribution, such as top-k candidates together with their relative ordering. It may also apply when such knowledge is not directly provided but can be inferred from indirect access, such as by using repeated samples and statistical testing to recover coarse distributional properties with high probability. Finally, weakly asymmetric steganography provides a useful basis for settings where exact symmetry between the sender's and receiver's distributions is difficult to maintain, for example due to numerical precision, hardware, or implementation differences. In these cases, relying on partial knowledge can be more robust than requiring exact probability agreement, while still exploiting more information than fully asymmetric schemes. It may also serve as a starting point for more practical designs that further improve reliability under such imperfect conditions.

7 Conclusion

We introduced weakly asymmetric steganography, a new model that captures settings where the receiver possesses partial channel knowledge. We showed that such partial knowledge can be explicitly exploited by constructing provably secure steganography schemes under representative knowledge, including threshold-based knowledge in binary alphabets and rank-based knowledge in general alphabets. Our experimental results in language-model-based text steganography further suggest the potential of this approach under restricted receiver access and distribution mismatch.

Acknowledgments

This work was supported by the National Natural Science Foundation of China under Grant Nos. U2336208 and 12025104.

Ethical Considerations

This section outlines the ethical considerations of our research, following a stakeholder-based analysis.

Stakeholder Analysis and Process Impact Our stakeholders include the research team, future researchers, potential users of steganographic systems, security professionals, and the public. The research team benefits from a clearer formalization of weakly asymmetric steganography, but also bears responsibility for communicating its assumptions, limitations, and dual-use implications accurately. Future researchers may benefit from a more precise model for studying partial-knowledge steganography, but this model could also inform the design of stronger covert channels. Potential users of steganographic systems, including benign users who rely on private communication, may benefit from better-understood security guarantees, but could be harmed if they overestimate the practical protection provided by theoretical schemes. Security professionals may benefit from a more accurate characterization of steganographic capabilities under partial knowledge, which can support defensive analysis and policy discussion, but may face increased challenges if such techniques are adapted faster than corresponding defenses. The public may benefit from improved scientific understanding and better-informed security governance, but could be harmed if the resulting ideas are used for illicit, illegal, or harmful activities.

The research process itself posed limited direct risk. The work is primarily theoretical: we formalize a model, prove security/correctness properties, and evaluate controlled instantiations using public models and synthetic generations. We did not collect personal data, involve human subjects, or interact with real users, production systems, or sensitive infrastructure.

Impact of the Publication The publication has both positive and negative implications. On the positive side, it gives the community a more realistic framework for reasoning about covert communication when receivers have partial, rather than full or zero, channel knowledge. This can improve both scientific understanding and defensive analysis by clarifying what kinds of partial access may already be enough to enable nontrivial steganography. On the negative side, like most steganography research, it could inform the design of more practical covert channels.

Mitigation To mitigate this risk, we frame the contribution as foundational analysis rather than a deployment recipe; we explicitly state the assumptions and limitations of the weakly asymmetric model so its practical utility is not overstated; and we evaluate only controlled, public-resource instantiations.

Justification for Research We believe conducting and publishing this work is justified because a precise understanding of the capabilities and limits of partial-knowledge steganography is necessary for both future defensive research and informed policy discussion. Any real-world deployment should undergo additional context-specific risk assessment and safeguards.

Open Science

The artifact accompanying this paper is publicly available at: https://doi.org/10.5281/zenodo.20428322

The repository provides implementations of our rank-based weakly asymmetric steganography schemes, together with scripts for running the Section 5 experiments for these schemes.

References

[1] Michael Backes and Christian Cachin. Public-key steganography with active attacks. In *Theory of Cryptography, Second Theory of Cryptography Conference, TCC 2005, Cambridge, MA, USA, February 10-12, 2005, Proceedings*, Lecture Notes in Computer Science, pages 210–226. Springer, 2005.

[2] Minhao Bai, Jinshuai Yang, Kaiyi Pang, Xin Xu, Zhen Yang, and Yongfeng Huang. Provably robust and secure steganography in asymmetric resource scenario. In *IEEE Symposium on Security and Privacy, SP 2025, San Francisco, CA, USA, May 12-15, 2025*, pages 1438–1456. IEEE, 2025.

[3] Sebastian Berndt and Maciej Liskiewicz. Provable secure universal steganography of optimal rate: Provably secure steganography does not necessarily imply one-way functions. In *Proceedings of the 4th ACM Workshop on Information Hiding and Multimedia Security, IH&MMSec 2016, Vigo, Galicia, Spain, June 20-22, 2016*, pages 81–92. ACM, 2016.

[4] Sebastian Berndt and Maciej Liskiewicz. On the gold standard for security of universal steganography. In *Advances in Cryptology - EUROCRYPT 2018 - 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29 - May 3, 2018 Proceedings, Part I*, Lecture Notes in Computer Science, pages 29–60. Springer, 2018.

[5] Manuel Blum and Silvio Micali. How to generate cryptographically strong sequences of pseudo random bits. In *23rd Annual Symposium on Foundations of Computer Science, Chicago, Illinois, USA, 3-5 November 1982*, pages 112–117. IEEE Computer Society, 1982.

[6] Dan Boneh and Victor Shoup. *A Graduate Course in Applied Cryptography*. 2023. Version 0.6, January 2023. URL: https://toc.cryptobook.us/.

[7] Mark Chen, Alec Radford, Rewon Child, Jeffrey Wu, Heewoo Jun, David Luan, and Ilya Sutskever. Generative pretraining from pixels. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, Proceedings of Machine Learning Research, pages 1691–1703. PMLR, 2020.

[8] Miranda Christ and Sam Gunn. Pseudorandom error-correcting codes. In *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part VI*, Lecture Notes in Computer Science, pages 325–347. Springer, 2024.

[9] Miranda Christ, Sam Gunn, and Or Zamir. Undetectable watermarks for language models. In *The Thirty Seventh Annual Conference on Learning Theory, June 30 - July 3, 2024, Edmonton, Canada*, Proceedings of Machine Learning Research, pages 1125–1139. PMLR, 2024.

[10] Nenad Dedic, Gene Itkis, Leonid Reyzin, and Scott Russell. Upper and lower bounds on black-box steganography. *J. Cryptol.*, 22(3):365–394, 2009.

[11] Jinyang Ding, Kejiang Chen, Yaofei Wang, Na Zhao, Weiming Zhang, and Nenghai Yu. Discop: Provably secure steganography in practice based on "distribution copies". In *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*, pages 2238–2255. IEEE, 2023.

[12] Phillipa Gill, Masashi Crete-Nishihata, Jakub Dalek, Sharon Goldberg, Adam Senft, and Greg Wiseman. Characterizing web censorship worldwide: Another look at the opennet initiative data. *ACM Trans. Web*, 9(1):4:1–4:29, 2015.

[13] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, et al. The llama 3 herd of models. *CoRR*, abs/2407.21783, 2024. arXiv:2407.21783.

[14] Horace He and Thinking Machines Lab. Defeating nondeterminism in llm inference. *Thinking Machines Lab: Connectionism*, 2025. URL: https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/.

[15] Nicholas J. Hopper. *Toward a Theory of Steganography*. Ph.D. dissertation, technical report CMU-CS-04-157, Carnegie Mellon University, Pittsburgh, PA, USA, July 2004.

[16] Nicholas J. Hopper, John Langford, and Luis von Ahn. Provably secure steganography. In *Advances in Cryptology - CRYPTO 2002, 22nd Annual International Cryptology Conference, Santa Barbara, California, USA, August 18-22, 2002, Proceedings*, Lecture Notes in Computer Science, pages 77–92. Springer, 2002.

[17] Gabriel Kaptchuk, Tushar M. Jois, Matthew Green, and Aviel D. Rubin. Meteor: Cryptographically secure steganography for realistic distributions. In *CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021*, pages 1529–1548. ACM, 2021.

[18] Tri Van Le. Efficient provably secure public key steganography. *IACR Cryptol. ePrint Arch.*, 2003:156, 2003.

[19] Guorui Liao, Jinshuai Yang, Weizhi Shao, and Yongfeng Huang. A framework for designing provably secure steganography. In *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13-15, 2025*, pages 6837–6856. USENIX Association, 2025.

[20] Anna Lysyanskaya and Mira Meyerovich. Provably secure steganography with imperfect sampling. In *Public Key Cryptography - PKC 2006, 9th International Conference on Theory and Practice of Public-Key Cryptography, New York, NY, USA, April 24-26, 2006, Proceedings*, Lecture Notes in Computer Science, pages 123–139. Springer, 2006.

[21] OpenAI. GPT-4 technical report. *CoRR*, abs/2303.08774, 2023. arXiv:2303.08774.

[22] Ram Sundara Raman, Prerana Shenoy, Katharina Kohls, and Roya Ensafi. Censored planet: An internet-wide, longitudinal censorship observatory. In *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*, pages 49–66. ACM, 2020.

[23] Luis von Ahn and Nicholas J. Hopper. Public-key steganography. In *Advances in Cryptology - EUROCRYPT 2004, International Conference on the Theory and Applications of Cryptographic Techniques, Interlaken, Switzerland, May 2-6, 2004, Proceedings*, Lecture Notes in Computer Science, pages 323–341. Springer, 2004.

[24] Yaofei Wang, Gang Pei, Kejiang Chen, Jinyang Ding, Chao Pan, Weilong Pang, Donghui Hu, and Weiming Zhang. Sparsamp: Efficient provably secure steganography based on sparse sampling. In *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13-15, 2025*, pages 6817–6835. USENIX Association, 2025.

[25] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *CoRR*, abs/2412.15115, 2024. arXiv:2412.15115.

[26] Or Zamir. Undetectable steganography for language models. *Trans. Mach. Learn. Res.*, 2024.

A Examples of Generated Stegotexts

This appendix provides examples of stegotexts generated using Rank(Full). The original outputs were in Markdown format; for readability, we converted the Markdown syntax into standard LaTeX formatting without changing the textual content.

Prompt

Tell me something about steganography

Llama 3

Steganography! It's an intriguing field that's both fascinating and potentially mischievous.
Steganography is the art and science of hiding secret messages, images, audio, or video within plain sight, where the hidden content is not immediately detectable. The goal is to conceal the existence of the hidden message, making it impossible for unauthorized parties to detect or intercept it.
Steganography has a long history, dating back to ancient civilizations, where it was used to conceal secret messages and military communications. In modern times, steganography has become popular in various domains, including:

1. **Digital watermarking**: Watermarks are embedded in digital images, audio, or video to prove ownership, track usage, or detect tampering.
2. **Data hiding**: Files, documents, or messages are encoded within seemingly innocuous data, such as images, emails, or text files.
3. **Image steganography**: Hidden information is embedded within digital images, often using techniques like frequency domain analysis or spatial domain encoding.
4. **Audio steganography**: Secret messages are concealed within audio files, using techniques like echo hiding, frequency masking, or spread-spectrum coding.

Steganography can be used for various purposes, including:

1. **Secure communication**: Steganography can provide an additional layer of security for sensitive information.
2. **Copyright protection**: Watermarks can be used to prove ownership and track unauthorized use.
3. **Forensic analysis**: Steganographic techniques can help investigators detect and reconstruct hidden evidence in digital forensic cases.

4. **Art and entertainment**: Steganography can be used to create interactive and engaging multimedia experiences.

However, steganography also has its limitations and potential drawbacks:

1. **Detectability**: Steganographic methods can be detectable using advanced analysis techniques, such as machine learning-based tools.
2. **Capacity limitations**: The amount of information that can be embedded within a host media (e.g., image or audio) is limited.
3. **Error correction**: Since steganographic data is hidden, errors during transmission or storage can be difficult to correct.
4. **Legal implications**: The use of steganography for unauthorized purposes, such as hiding illegal content, can lead to legal consequences.

In conclusion, steganography is an exciting and rapidly evolving field that offers new possibilities for secure communication, creative expression, and data protection. However, it's essential to recognize the ethical and legal implications of using steganographic techniques and to ensure that they are used responsibly.

Qwen 2.5

Steganography is the practice of hiding secret data within an ordinary, non-secret file or message to avoid detection. The term "steganography" comes from the Greek words "stegano-" (meaning covered or concealed) and "-graphy" (meaning writing). Unlike cryptography, which encrypts data to make it unreadable to unauthorized parties, steganography aims to keep the secret information hidden in plain sight on the surface of a digital file, image, or message.
Here are some key aspects of steganography:

1. **Types of Data Hidden**: Steganography can hide various types of data, including text, images, video, audio, and network packets. Common examples include hidden text inside an image or a video frame, where the human eye can't immediately spot the alteration.

2. **Media Used**: While the most common media used for steganography are images, other media such as audio files, video files, and even executable files can also be used to hide data.

3. **Common Techniques**:
    - **Least Significant Bit (LSB) Replacement**: This is a common technique in digital steganography. It involves replacing the least significant bits of a pixel's color value with the bits of the secret message, without noticeably altering the appearance of the image to the naked eye.
    - **Insertion in Metadata**: Steganographic data can also be embedded in the metadata of a file, such as EXIF data in images or MP3 tags in audio files.
    - **Splicing Techniques**: This involves splitting the cover material into parts and redistributing them to hide data. For example, a secret message can be embedded by altering the last bit of specific bytes or even entire blocks of data.

4. **Masking and Watermarking**:
    - **Masking**: This technique involves obscuring the embedded data using another type of cover data to make the hidden information less detectable.
    - **Watermarking**: Similar to steganography, watermarking involves embedding a signal into a cover medium, but the purpose of watermarking is usually to detect or identify the presence of the signal rather than keeping it a secret.

5. **Detection and Analysis**: There are tools and methods to detect steganography, such as statistical analysis and machine learning techniques. However, more advanced forms of steganography can be difficult to detect without specialized knowledge.

6. **Legal and Ethical Considerations**: The use of steganography can have legal and ethical implications. It is often used for benign purposes like data transfer, but it can also be used for malicious activities such as spreading malware or hiding illegal content.

Steganography plays a significant role in cybersecurity and digital forensics. As technology advances, so do the methods and techniques used in both steganography and its detection.